FROM REACTIVE TO CONVERSATIONAL

Building a Production AIOps Observability System

Phases 18–20: Predict → Explain → Remember & Verify

PHASE 18	PHASE 19	PHASE 20
PREDICT	**EXPLAIN**	**REMEMBER & VERIFY**
"I see the future"	*"I understand the past"*	*"I hold the thread"*

Michel Mazza | HomeLab Observability Trilogy | April 2026

⚠️ INFRASTRUCTURE OBSERVABILITY AT SCALE

IT'S 11PM. A METRIC IS TRENDING UP. YOU HAVE NO IDEA WHY.

- Thousands of metric data points per minute
- Existing tools show WHAT — not WHY
- Operators still do correlation manually
- "Is CPU still high?" = 4 dashboards

THE GAP

METRICS
CPU 87% | Mem ↑ | I/O ↑

THE GAP
(manual correlation)

DIAGNOSIS
Root cause + context

The machine does the measurement. The human does the correlation.

THE OBSERVABILITY TRILOGY

Three phases. Three capabilities. One coherent system.

PHASE 18

PREDICT

"CPU on node 1 will hit threshold in 47 min."

`OLS regression over live Prometheus`

PHASE 19

EXPLAIN

"Caused by service restart 45s before spike."

`Nomad/Consul events + multiplicative scoring`

PHASE 20

REMEMBER & VERIFY

"Still related to that restart? Status: increasing."

`DiagnosticContext + Verifiable Inference`

"The system sees the future, understands the past, and maintains the diagnostic thread over time."

PHASE 18: ZERO-LLM OLS REGRESSION

Classic statistics. Not machine learning.

WHAT IT DOES

- OLS regression over 9 live Prometheus series
- Time-to-Threshold (TTA) forecasting
- R^2 color-coded confidence gating
- Observer Effect: self-aware resource flag

P95 LATENCY

~0.05ms

100× under 5ms budget

Deterministic • Repeatable • Bounded

Impossible with an LLM in the hot path

THE FOUNDING CONSTRAINT: No LLM in the analytical path. OLS gives deterministic, latency-bounded analytics that shaped every design decision that followed.

PHASE 19: WHY IS CPU HIGH?

Answered in <10ms. Zero LLM in the analytical path.

PIPELINE

EVENT COLLECTION

Nomad delta-poll
Consul diff-poll (30s)

▶

INFLECTION DETECT

step_change +
trend_shift fallback

▶

MULTIPLICATIVE

recency ×
severity × node

▶

TOP CAUSE

4-state result
in <10ms

FOUR DIAGNOSTIC STATES

Candidate

Score + attribution

Unknown Cause

Events exist, no match

No Events Logged

Collector has no data

Collector Gap

Collector offline — honest

> *Collector Gap is the system being honest about what it doesn't know.*

THE STATELESS ORACLE PROBLEM

Phases 18 and 19 predict and explain — then forget everything.

BEFORE: STATELESS

```
Q:
Why is CPU high?
A:
Service restart on node 3

Q: (90s later)
Is it still related?
A:
I don't know what you mean.
```

AFTER: THREADED

```
Q:
Why is CPU high?
A:
Service restart on node 3

Q: (90s later)
Is it still related?
A:
Yes. Current: INCREASING (12s ago)
```

A system that recites historical data as if it were current truth.

THE CONTEXT WINDOW GUARD

The only sanctioned path from diagnostic data to the LLM prompt.

`correlate()`
Phase 19 engine

▶

`DiagnosticContext`
7 structured fields

▶

GUARD
≤200 char hard cap

▶

`LLM prompt`
Synthesis only

DUAL-LAYER TRUNCATION

- Field-level: caps each structured field
- String-level: safety net on final output
- 200-char invariant guaranteed
- Future-proof against template drift

`TTL = 300s`

THE HUMAN DELIBERATION BUFFER

Environment-configurable.
Stale context auto-cleared.

`diagnostic_context_to_summary()` — the only function in the codebase that touches the LLM prompt

VERIFIABLE INFERENCE

MEMORY WITHOUT HALLUCINATION

FOLLOW-UP FLOW

FOLLOW-UP QUERY

"Is it still related?"

CACHED CONTEXT

DiagnosticContext retrieved

LIVE TREND CHECK

TrendMonitor polled

RESPONSE

Cached + live status

"Yes — still related to the service restart. Current status: INCREASING as of 12 seconds ago."

GUARANTEES

New-query override

Diagnostic signals win over follow-up

Graceful degradation

Trend failure never blocks follow-up

Zero stale responses

Verified in smoke test suite

TAB 8: THE DIAGNOSTIC THREAD

How the system thinks — not just what it answers.

TAB 8 ▸ Diagnostic Thread (append-only)

14:32:18 score 0.87
cpu_rate_node3
nomad: job_restart (auth-svc)

14:29:45 score 0.71
mem_avail_node1
consul: deregister (cache)

14:22:03
cpu_rate_node3
— investigation start —

KEY PROPERTIES

Append-only
Immutable timeline

Reverse-chronological
Newest first

Pivot Event
Auto-detected

Empty state
"No investigations"

DAY 2 OPERATIONS MINDSET

Three tiers of infrastructure debt cleared before feature work.

HEARTBEAT CONTRACT

"ok" heartbeat proves collector healthy AND infra quiet. Distinguishes silence from outage.

LIFECYCLE MANAGER

Atomic 14-day/7-day retention. Both tables pruned or neither. Nomad periodic job @ 02:00 UTC.

WAL MODE PRAGMA

All 5 SQLite connections consistent. Prevents reader-writer blocking across scraper, collector, RAG.

4-NODE NOMAD CLUSTER

3 Proxmox VMs + ML workstation. Scraper migrated from launchd to cluster-managed raw_exec.

Operational properties that make a system trustworthy at 2am.

5 CONSECUTIVE PHASES. ZERO LLM IN ANALYTICAL PATH.

The LLM is still there — for natural language synthesis, not analytical decisions.

P16	P17	P18	P19	P20
Self-Awareness	Conflict Detection	Prediction	Correlation	Memory & Verify
A+	A+	A+	A+	A+

WHY IT MATTERS

Latency

OLS at 0.05ms. Impossible with LLM.

Consistency

Same input → same output. Always.

Testability

Unit tests for arithmetic, not LLM output.

999 tests **980/980** runnable @ 100% **ZERO** regressions across 9 sessions

THE DEMO: ASK IT TWICE

Two questions. That's the demo.

Q1 **"Why is CPU high?"**

- Intent classified: diagnostic
- Correlation engine: Prometheus inflections × Nomad/Consul events
- Top cause surfaced in Tab 7
- **Answered in <10ms**

Q2 **"Is it still related to that restart?" (90s later)**

- Follow-up phrase recognized
- DiagnosticContext retrieved from cache
- Live TrendMonitor polled
- Response: ***"Current status: INCREASING as of 23s ago"***

The diagnosis came from the correlation engine. The verification came from the trend monitor. The LLM rendered it.

ARCHITECTURE: THE FULL STACK

Predict → Explain → Remember → Verify

LAYER 1: INGEST

Prometheus scraper (Nomad job) → metric_snapshots | Event collector → log_events + heartbeats

LAYER 2: ANALYTICAL

OLS Engine (Phase 18) ◆ Correlation Engine + multiplicative scoring (Phase 19)

LAYER 3: MEMORY & GUARD

DiagnosticContext (7 fields) → diagnostic_context_to_summary() guard → ≤200 chars → LLM prompt

LAYER 4: UI

Glass Box Tab 7 (predictions + top cause) ◆ Tab 8 (diagnostic thread)

Every component observable. Every decision traceable. Every analytical step deterministic.

WHAT THIS DEMONSTRATES

ENGINEERING DISCIPLINE

999 tests, zero regressions, 5-phase zero-LLM streak. Not an accident — a process.

SYSTEM THINKING

Heartbeat contract, atomic retention, WAL consistency. Trustworthy at 2am.

CONVERSATIONAL DESIGN

Context guard contract, follow-up detection, graceful degradation. Behavior at the edges.

PRODUCTION MINDSET

Cluster-managed jobs, configurable TTL, dry-run mode. Day 2 thinking, not just demo.

"A definitive shift from building a 'tool' to engineering a 'system.'"

— Gemini, Phase 20 completion assessment